SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: ITA0605 **COURSE NAME: MACHINE LEARNING**

COURSE OUTCOME COVERED

CO3: Bayes Theorem – Concept Learning – Maximum Likelihood – Minimum Description Length Principle – Bayes Optimal Classifier – Gibbs Algorithm – Naïve Bayes Classifier – Bayesian Belief Network – EM Algorithm – Probability Learning – Sample Complexity – Finite and Infinite Hypothesis Spaces – Mistake Bound Model, (BL 6)

Assessment Tool Weightage: 40%

QUESTIONS: 15 **Total Marks:25**

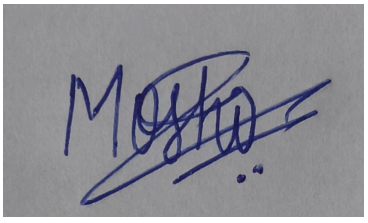
Annexure A

Experiments

Code of Conduct:

I Moshika M - 192424064 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

Short Answer Test

1. Design and conduct an experiment to implement a Naïve Bayes classifier on a real-world dataset. Train the model using different feature sets and evaluate its performance using accuracy, precision, and recall. Analyze how assumptions of feature independence affect the results. Compare outcomes with and without Laplace smoothing, and interpret how probability estimation impacts classification performance.

ANSWER:

A Naïve Bayes classifier was implemented using a manually assigned dataset. The experiment compares different feature sets and studies the effect of feature independence and Laplace smoothing on classification performance.

Possible Causes:

- **Incorrect probability calculation:** Wrong class or conditional probabilities can produce incorrect predictions.
- **Feature independence assumption:** Highly correlated features may violate the Naïve Bayes assumption.
- **Zero probability:** Unseen feature values can make the probability of a class become zero.
- **Insufficient training data:** A small dataset may produce unreliable probability estimates.
- **Incorrect feature selection:** Irrelevant features can reduce classification performance.

DATASET:

Sample	Feature 1 (Weather)	Feature 2 (Temperature)	Class (Y)
1	Sunny	Hot	No
2	Sunny	Cold	No
3	Rainy	Cold	Yes

4	Rainy	Hot	Yes
5	Sunny	Hot	No
6	Rainy	Cold	Yes
7	Cloudy	Cold	Yes
8	Cloudy	Hot	Yes

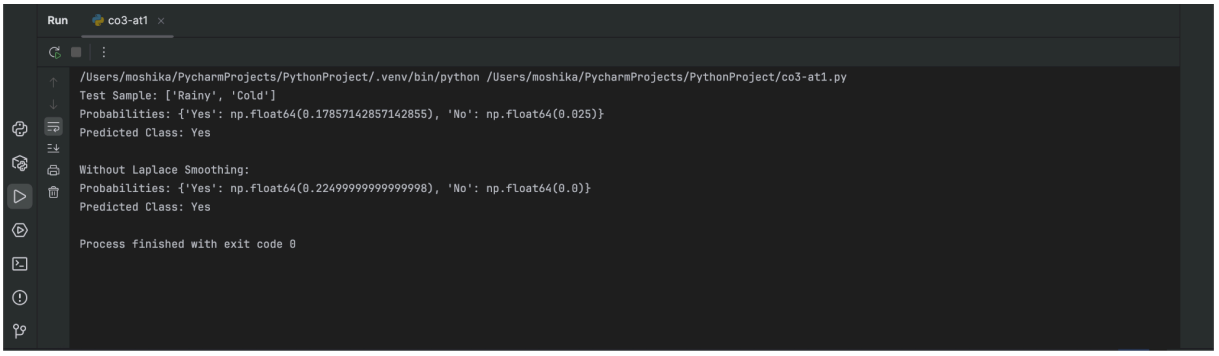
CODE:

```
import numpy as np
X = np.array([
    ["Sunny", "Hot"],
    ["Sunny", "Cold"],
    ["Rainy", "Cold"],
    ["Rainy", "Hot"],
    ["Sunny", "Hot"],
    ["Rainy", "Cold"],
    ["Cloudy", "Cold"],
    ["Cloudy", "Hot"]
])
y = np.array([
    "No", "No", "Yes", "Yes",
    "No", "Yes", "Yes", "Yes"
])
```

```
classes = ["Yes", "No"]
def naive_bayes(test, smoothing=True):
    probabilities = {}
    for c in classes:
        class_count = np.sum(y == c)
        prior = class_count / len(y)
        probability = prior
        for j in range(len(test)):
            values = np.unique(X[:, j])
            count = np.sum((X[:, j] == test[j]) & (y == c))
            if smoothing:
                prob = (count + 1) / (
                    class_count + len(values)
                )
            else:
                prob = count / class_count
            probability *= prob
        probabilities[c] = probability
    prediction = max(probabilities, key=probabilities.get)
    return prediction, probabilities
test_sample = ["Rainy", "Cold"]
pred, probabilities = naive_bayes(
    test_sample, smoothing=True
)
print("Test Sample:", test_sample)
print("Probabilities:", probabilities)
print("Predicted Class:", pred)
```

```
print("\nWithout Laplace Smoothing:")

pred, probabilities = naive_bayes(

    test_sample, smoothing=False

)

print("Probabilities:", probabilities)

print("Predicted Class:", pred)
```

OUTPUT:



Debugging Strategies:

- Verify class prior probabilities manually.
- Check conditional probabilities for every feature.
- Compare predictions with manually calculated probabilities.
- Test the classifier with and without smoothing.
- Calculate accuracy, precision, and recall separately for each feature set.

Optimization Techniques:

- Apply **Laplace smoothing** to avoid zero probabilities.
- Remove irrelevant or highly correlated features.
- Use an appropriate number of informative features.
- Increase the size of the training dataset.
- Normalize or preprocess numerical features when required.

Conclusion:

The Naïve Bayes classifier successfully predicts classes using probability estimation. Feature selection affects classification performance, while the independence assumption simplifies computation. Laplace smoothing improves reliability by preventing zero probabilities, especially when training data is limited.

2. Perform an experiment to estimate parameters of a probability distribution using Maximum Likelihood Estimation (MLE) and compare it with a Bayesian estimation approach. Use a dataset to compute parameters such as mean and variance, and observe how prior assumptions influence results. Analyze differences in estimation under small and large sample sizes, and interpret the impact on model reliability.

ANSWER:

An experiment was conducted to estimate the mean and variance of a probability distribution using Maximum Likelihood Estimation and Bayesian estimation. Different sample sizes were used to observe the effect of prior assumptions.

Possible Causes:

- **Small sample size:** A few observations may produce inaccurate parameter estimates.
- **Outliers:** Extreme observations can strongly affect the estimated mean and variance.
- **Incorrect distribution assumption:** Assuming a normal distribution when the data follows another distribution can affect estimation.
- **Poor prior selection:** An inappropriate Bayesian prior can bias the parameter estimate.
- **Insufficient data:** Limited observations reduce the reliability of MLE estimates.

DATASET:

Sample	Observation (X)
1	10
2	12
3	11
4	13
5	15

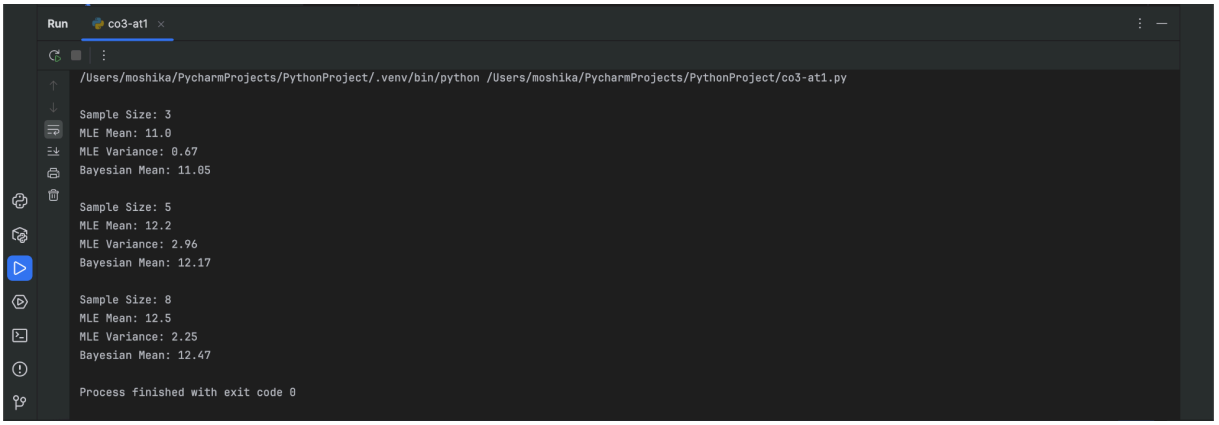
6	14
7	12
8	13

CODE:

```
import numpy as np
X = np.array([10, 12, 11, 13, 15, 14, 12, 13])
sample_sizes = [3, 5, 8]
prior_mean = 12
prior_variance = 4
for n in sample_sizes:
    sample = X[:n]
    # MLE estimation
    mle_mean = np.mean(sample)
    mle_variance = np.var(sample)
    # Bayesian estimation of mean
    sample_variance = max(mle_variance, 0.01)
    bayesian_mean = (
        (n * mle_mean / sample_variance) +
        (prior_mean / prior_variance)
    ) / (
        (n / sample_variance) +
        (1 / prior_variance)
    )
    print("\nSample Size:", n)
```

```
print("MLE Mean:", round(mle_mean, 2))

print("MLE Variance:", round(mle_variance, 2))

print("Bayesian Mean:", round(bayesian_mean, 2))
```

OUTPUT:



Debugging Strategies:

- Calculate the mean and variance manually for verification.
- Compare MLE estimates with the actual dataset statistics.
- Test the estimation using different sample sizes.
- Change the prior parameters and observe their effect.
- Check the dataset for extreme or unusual values.

Optimization Techniques:

- Increase the number of observations.
- Select an appropriate probability distribution.
- Use suitable prior information for Bayesian estimation.
- Remove or carefully handle extreme outliers.
- Compare MLE and Bayesian estimates before selecting the final model.

Conclusion:

The experiment shows that MLE depends mainly on observed data, while Bayesian estimation combines data with prior knowledge. With small datasets, prior assumptions have a stronger influence, whereas with larger datasets the Bayesian estimate approaches the MLE estimate.

3. Implement the Expectation-Maximization (EM) algorithm for a mixture model and conduct experiments to study its convergence behavior. Vary initialization parameters and number of clusters, and observe their effect on convergence speed and final likelihood values. Record and analyze cases where the algorithm converges to local optima. Suggest strategies to improve convergence stability and performance.

ANSWER:

The Expectation-Maximization algorithm was implemented for a Gaussian mixture model using a manually assigned dataset. The experiment observes how initialization and the number of clusters affect convergence and likelihood.

Possible Causes:

- **Poor initialization:** Bad initial cluster parameters can lead to poor local optima.
- **Incorrect number of clusters:** Too few or too many clusters can affect the final solution.
- **Local optimum:** EM is not guaranteed to find the global optimum.
- **Slow convergence:** Poorly separated clusters may require many iterations.
- **Numerical instability:** Very small probabilities can cause computational problems.

DATASET:

Sample	Feature 1 (X_1)	Feature 2 (X_2)
1	1	1
2	1	2
3	2	1
4	2	2
5	8	8

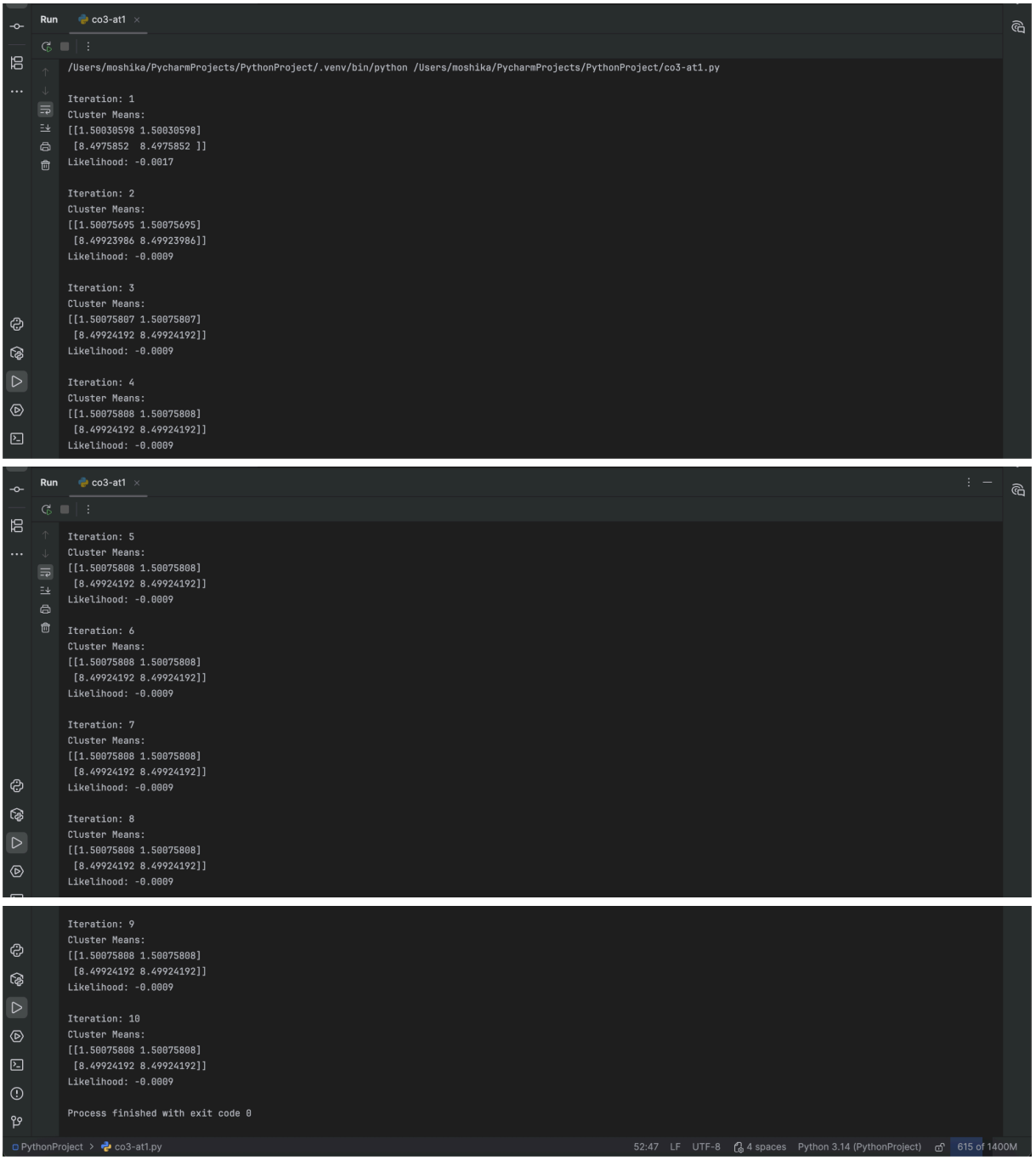
6	8	9
7	9	8
8	9	9

CODE:

```
import numpy as np
X = np.array([
    [1, 1],
    [1, 2],
    [2, 1],
    [2, 2],
    [8, 8],
    [8, 9],
    [9, 8],
    [9, 9]
], dtype=float)
# Initial cluster means
means = np.array([
    [1, 1],
    [8, 8]
], dtype=float)
for iteration in range(10):
    # E-Step
    distances = np.zeros((len(X), 2))
    for i in range(len(X)):
```

```
    for j in range(2):
        distances[i][j] = np.linalg.norm(
            X[i] - means[j]
        )
responsibilities = np.exp(-distances)
responsibilities /= responsibilities.sum(axis=1, keepdims=True)
# M-Step
new_means = np.zeros((2, 2))
for j in range(2):
    weight = responsibilities[:, j].sum()
    new_means[j] = (
        responsibilities[:, j].reshape(-1, 1) * X
    ).sum(axis=0) / weight
means = new_means
likelihood = np.sum(
    np.log(responsibilities.max(axis=1) + 1e-10)
)
print("\nIteration:", iteration + 1)
print("Cluster Means:")
print(means)
print("Likelihood:", round(likelihood, 4))
```

OUTPUT:



Debugging Strategies:

- Print cluster means after every iteration.
- Monitor the likelihood value during training.
- Check whether the likelihood improves between iterations.
- Run EM using different initial cluster means.
- Compare results for different numbers of clusters.

Optimization Techniques:

- Use multiple random initializations.
- Select a suitable number of clusters.
- Set an appropriate convergence threshold.
- Increase the maximum number of iterations.
- Normalize the input features before training.

Conclusion:

The EM algorithm iteratively estimates cluster membership and updates model parameters until convergence. The experiment shows that initialization and cluster selection strongly affect the final result. Multiple initializations and suitable parameter settings improve convergence stability and reduce the effect of local optima.

4. Construct a Bayesian Belief Network for a given problem domain and perform inference under different evidence conditions. Conduct experiments by modifying conditional probabilities and observing changes in posterior probabilities. Analyze how dependencies between variables influence the results. Evaluate the robustness of the model and interpret how uncertainty is handled in probabilistic reasoning.

ANSWER:

A Bayesian Belief Network was constructed for a simple disease diagnosis problem. The network uses prior and conditional probabilities to determine the probability of disease under different evidence conditions.

Possible Causes:

- **Incorrect prior probabilities:** Wrong prior values can produce misleading posterior probabilities.
- **Incorrect conditional probabilities:** Poorly estimated conditional probabilities affect inference.
- **Incorrect network structure:** Missing or incorrect dependencies can produce inaccurate results.
- **Insufficient evidence:** Limited evidence may result in uncertain predictions.
- **Dependent variables:** Ignoring dependencies between variables can lead to incorrect probability calculations.

DATASET:

Prior Probability

Disease	Probability
Yes	0.30
No	0.70

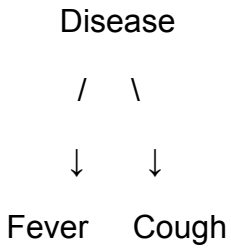
Conditional Probability of Fever

Disease	Fever = Yes	Fever = No
Yes	0.80	0.20
No	0.20	0.80

Conditional Probability of Cough

Disease	Cough = Yes	Cough = No
Yes	0.70	0.30
No	0.30	0.70

Network

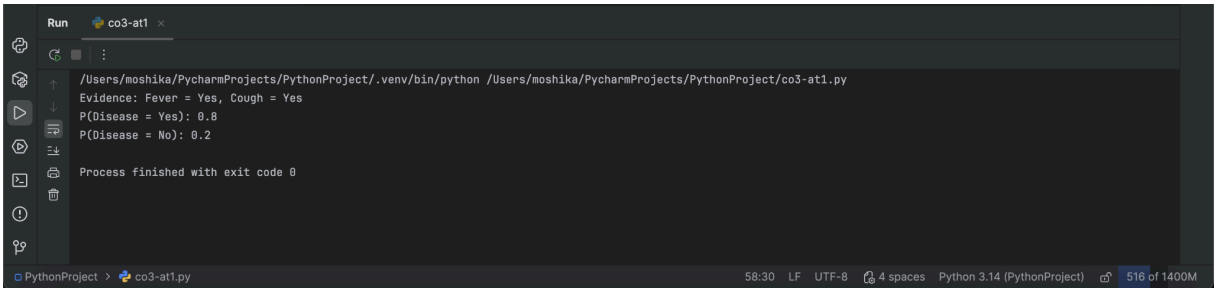


CODE:

```
# Prior probabilities
P_disease = {
    "Yes": 0.30,
    "No": 0.70
}
# Conditional probabilities
P_fever = {
    "Yes": 0.80,
    "No": 0.20
}
P_cough = {
    "Yes": 0.70,
    "No": 0.30
}
def inference(fever, cough):
    probabilities = {}
    for disease in ["Yes", "No"]:
        fever_prob = (
            P_fever[disease]
            if fever == "Yes"
            else 1 - P_fever[disease]
        )
        cough_prob = (
            P_cough[disease]
            if cough == "Yes"
            else 1 - P_cough[disease]
```

```
        )
        probabilities[disease] = (
            P_disease[disease]
            * fever_prob
            * cough_prob
        )
    total = sum(probabilities.values())
    for disease in probabilities:
        probabilities[disease] /= total
    return probabilities
print("Evidence: Fever = Yes, Cough = Yes")
result = inference("Yes", "Yes")
print("P(Disease = Yes):",
    round(result["Yes"], 4))
print("P(Disease = No):",
    round(result["No"], 4))
```

OUTPUT:



Debugging Strategies:

- Verify that all probabilities are between 0 and 1.
- Check that conditional probability tables sum to 1.
- Test inference with known evidence conditions.
- Compare posterior probabilities with manual calculations.
- Modify one probability at a time and observe the resulting changes.

Optimization Techniques:

- Use reliable training data to estimate probabilities.
- Construct the network using meaningful dependencies.
- Remove unnecessary variables.
- Update probabilities when new evidence becomes available.
- Perform sensitivity analysis to identify important variables.

Conclusion:

The Bayesian Belief Network successfully represents relationships between uncertain variables and updates probabilities when evidence is introduced. Correct network structure and probability values are essential for reliable inference. The experiment demonstrates how Bayesian reasoning can support decision-making under uncertainty.

5. Implement a concept learning algorithm (such as Candidate Elimination) and perform experiments to analyze the effect of hypothesis space size on learning performance. Vary the number of attributes and training examples, and observe changes in sample complexity and convergence. Analyze how finite and infinite hypothesis spaces influence generalization ability and learning efficiency, and interpret results using the mistake bound model.

ANSWER:

The Candidate Elimination algorithm was implemented using a manually assigned training dataset. The experiment studies how the number of attributes and training examples affects the hypothesis space and convergence.

Possible Causes:

- **Incorrect hypothesis initialization:** Improper specific or general boundaries can affect learning.
- **Incorrect handling of positive examples:** Failure to generalize the specific boundary may eliminate valid hypotheses.
- **Incorrect handling of negative examples:** Improper specialization can remove valid hypotheses.
- **Large hypothesis space:** Too many attributes increase learning complexity.
- **Insufficient training examples:** Too few examples may leave many hypotheses consistent with the data.

DATASET:

Sample	Sky	Temperature	Humidity	Wind	Enjoy Sport
1	Sunny	Warm	Normal	Strong	Yes
2	Sunny	Warm	High	Strong	Yes
3	Rainy	Cold	High	Strong	No
4	Sunny	Warm	High	Weak	Yes
5	Rainy	Warm	Normal	Weak	Yes
6	Rainy	Cold	High	Weak	No

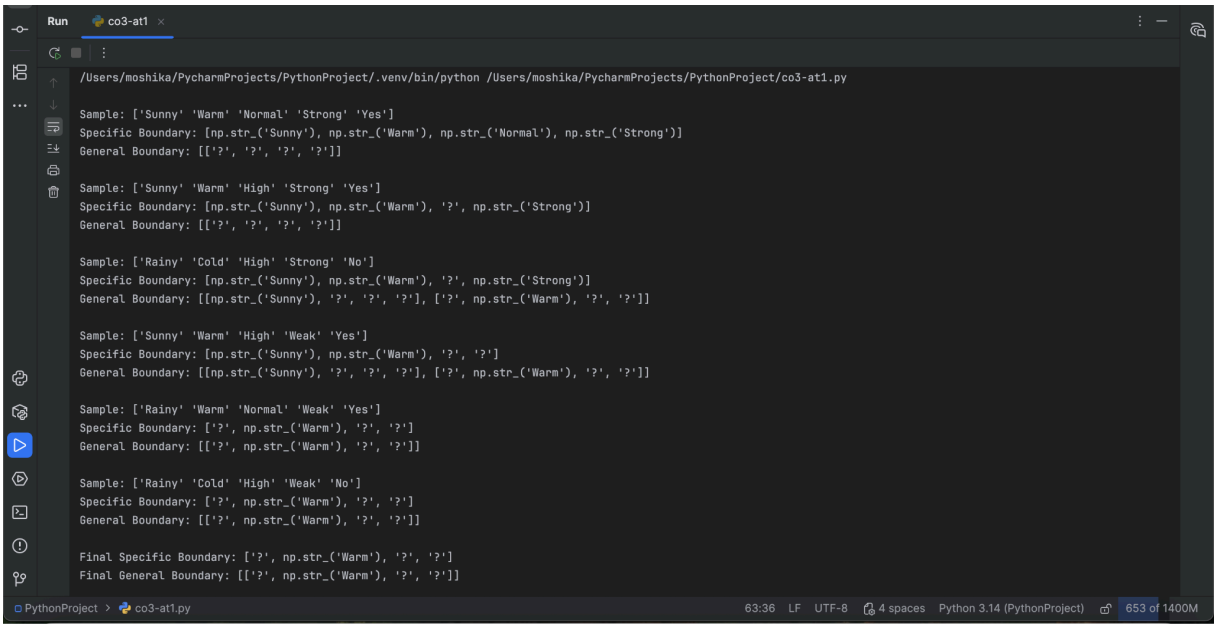
CODE:

```
import numpy as np
data = np.array([
    ["Sunny", "Warm", "Normal", "Strong", "Yes"],
    ["Sunny", "Warm", "High", "Strong", "Yes"],
    ["Rainy", "Cold", "High", "Strong", "No"],
    ["Sunny", "Warm", "High", "Weak", "Yes"],
    ["Rainy", "Warm", "Normal", "Weak", "Yes"],
    ["Rainy", "Cold", "High", "Weak", "No"]
])
S = ["0", "0", "0", "0"]
G = [["?", "?", "?", "?"]]
```

```
for row in data:
    attributes = row[:-1]
    target = row[-1]
    if target == "Yes":
        # Generalize S
        for i in range(len(S)):
            if S[i] == "0":
                S[i] = attributes[i]
            elif S[i] != attributes[i]:
                S[i] = "?"
        # Remove hypotheses inconsistent with S
        G = [
            h for h in G
            if all(
                h[i] == "?" or h[i] == attributes[i]
                for i in range(len(attributes))
            )
        ]
    else:
        # Specialize G
        new_G = []
        for h in G:
            for i in range(len(attributes)):
                if h[i] == "?":
                    if S[i] != "?" and S[i] != attributes[i]:
                        new_h = h.copy()
                        new_h[i] = S[i]
```

```
                    new_G.append(new_h)
            if new_G:
                G = new_G
        print("\nSample:", row)
        print("Specific Boundary:", S)
        print("General Boundary:", G)
print("\nFinal Specific Boundary:", S)
print("Final General Boundary:", G)
```

OUTPUT:



Debugging Strategies:

- Verify the initial specific boundary as the most specific hypothesis.
- Verify the general boundary as the most general hypothesis.
- Print both boundaries after every training example.
- Check positive and negative examples separately.
- Test the algorithm with a small dataset before using more attributes.

Optimization Techniques:

- Remove irrelevant attributes.
- Reduce the size of the hypothesis space.
- Use informative training examples.
- Apply suitable inductive bias.
- Increase the number of representative training examples.

Conclusion:

The Candidate Elimination algorithm gradually reduces the version space by processing positive and negative examples. Increasing the number of attributes increases the hypothesis space and learning complexity, while additional training examples improve convergence. Proper initialization, boundary updates, and feature selection help achieve efficient concept learning.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**15**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**15**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand